

GPU ARCHITECTURE

From Multi-GPU Hosts to Warps, Tensor Cores & CUDA Execution

A technical walkthrough: system topology, SM internals, memory hierarchy, asynchronous DMA transfers, and the CUDA programming model.

What We'll Cover



01

Multi-GPU Host Topology

How GPUs, CPUs, NVLink & PCIe fit together



02

GPU Chip Architecture

Streaming Multiprocessors (SMs) and HBM



03

Inside the SM

Threads, warps, scheduling, shared memory, ALUs, tensor cores



04

Async Memory & DMA

Copy engines moving data over NVLink and HBM

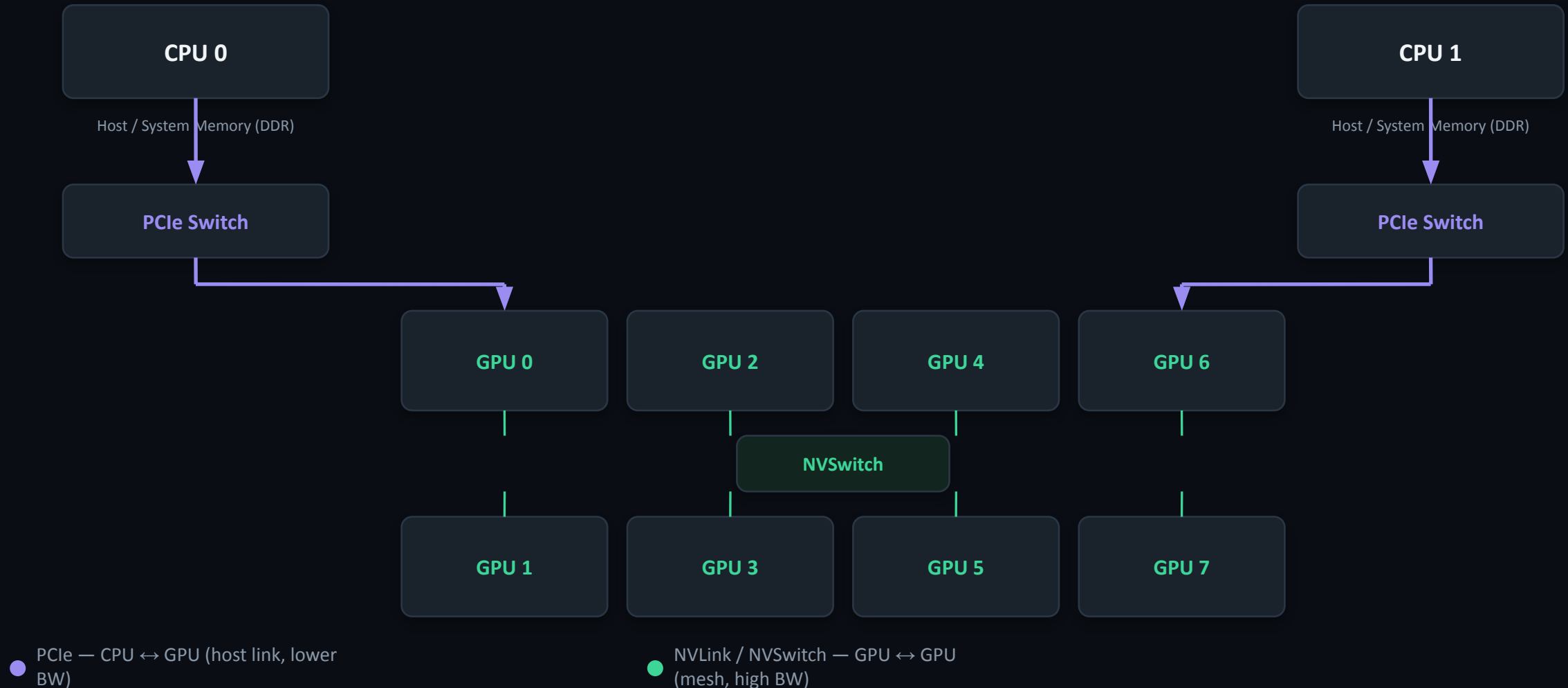


05

CUDA Programming Model

Blocks, threads, streams, context & synchronization

Multiple GPUs on a Single Host



Each GPU also has direct PCIe lanes to its socket for host ↔ device transfers (not all drawn for clarity).

NVLink vs. PCIe: Two Roads to the GPU



NVLink / NVSwitch

- Direct GPU ↔ GPU interconnect, purpose-built for scale-up
- H100: 18 links × ~26 GB/s ≈ 900 GB/s per GPU (bidirectional)
- NVSwitch lets every GPU talk to every other GPU at full bandwidth (all-to-all)
- Supports load/store, atomics & unified memory across GPUs
- Used for tensor/model/pipeline parallel traffic, all-reduce, NCCL collectives

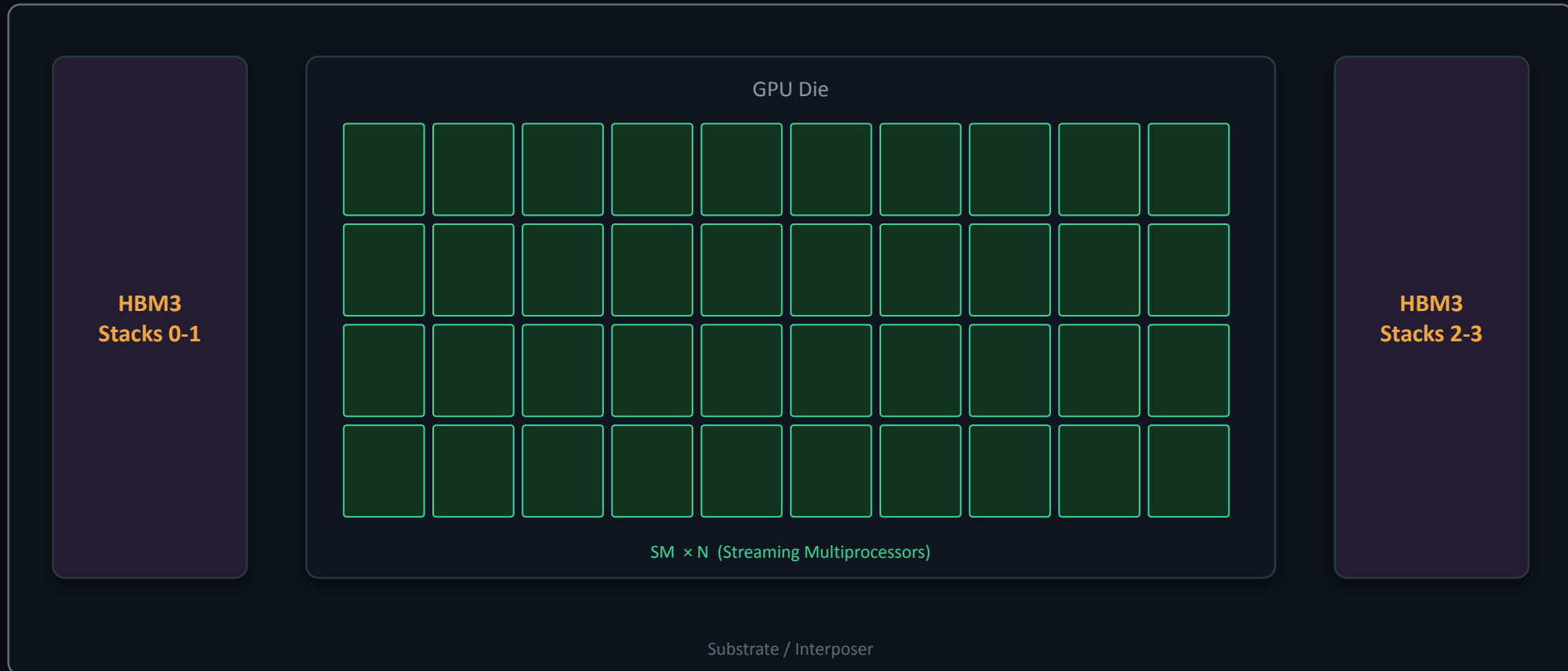


PCIe (Gen4/Gen5)

- General-purpose host ↔ device bus, shared with other peripherals
- PCIe Gen5 x16 ≈ 64 GB/s per direction (~128 GB/s bidirectional)
- Connects GPU to CPU, NICs, NVMe — the 'front door' to the host
- Higher latency, lower bandwidth than NVLink for GPU–GPU hops
- Used for host→device input staging, cross-node RDMA via NIC, control-plane traffic

Rule of thumb: NVLink scales GPUs together as one big accelerator; PCIe scales GPUs into the host.

GPU Die: SM Array + HBM Stacks



● SM array — parallel compute engines

● HBM stacks — high-bandwidth memory, stacked DRAM dies

HBM sits physically next to the die on a silicon interposer for ultra-wide, short data paths (thousands of I/O pins).

HBM: High Bandwidth Memory

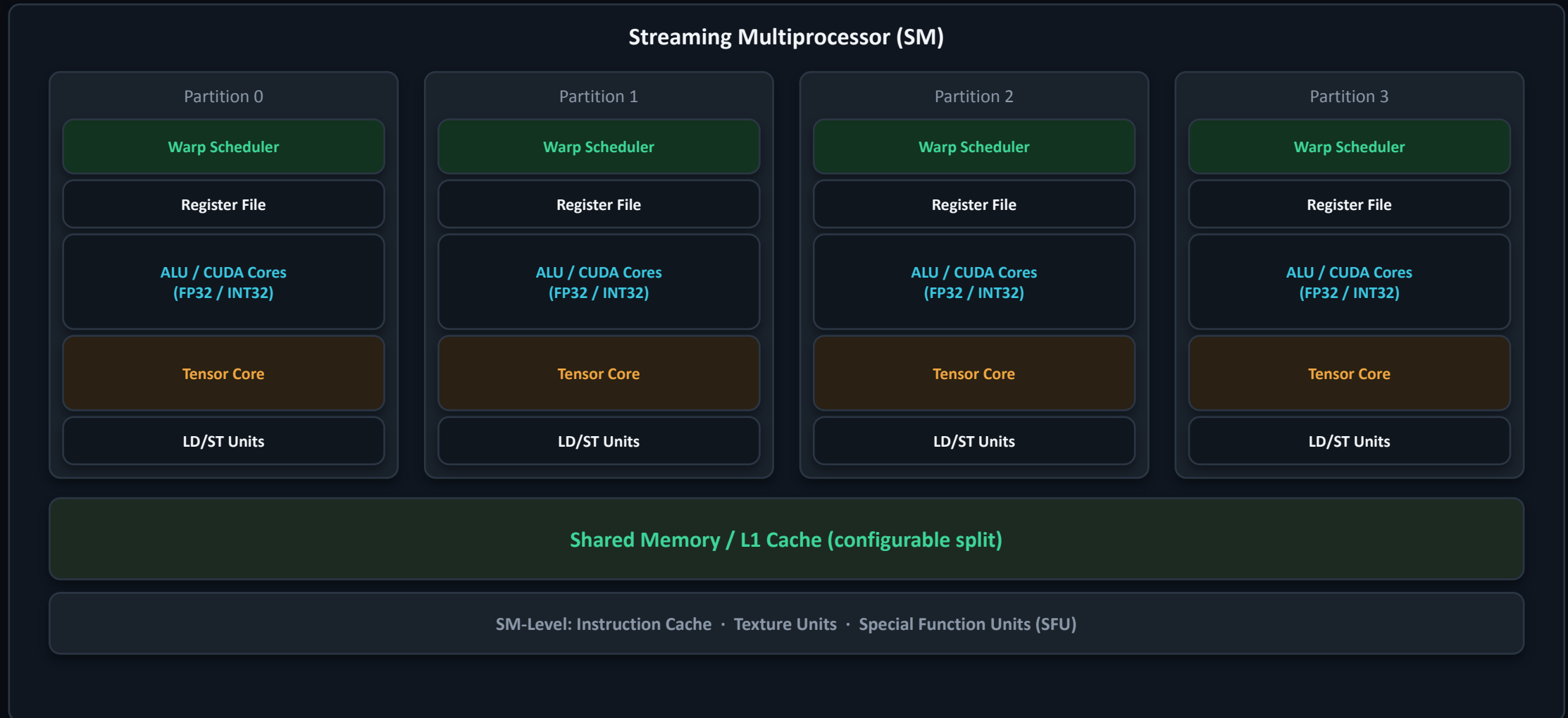


Through-Silicon Vias (TSVs)
connect stacked DRAM dies vertically

- DRAM dies stacked vertically (4–12 layers) and connected by thousands of TSVs
- Extremely wide bus (1024-bit per stack) run at moderate clock → massive bandwidth
- H100: 5 stacks $\approx$ 80 GB, $\sim$ 3.35 TB/s aggregate memory bandwidth
- Sits on the same interposer as the GPU die — short, wide, low-power datapath vs. off-package GDDR/DDR
- Feeds SMs through the GPU's L2 cache and memory controllers; bandwidth, not just capacity, is the bottleneck for most kernels

Why it matters: compute (FLOPs) has grown faster than memory bandwidth — many kernels are memory-bound, so HBM bandwidth often dictates real-world throughput.

Anatomy of a Streaming Multiprocessor



Threads → Warps: SIMT Execution

- A thread is the smallest unit of execution — runs one instance of the kernel
- The SM groups 32 consecutive threads into a warp — the real scheduling unit
- All 32 threads in a warp execute the same instruction in lockstep (SIMT)
- Divergent branches (if/else) serialize — each path runs with inactive lanes masked off
- Warps, not threads, are what the hardware scheduler actually tracks and issues

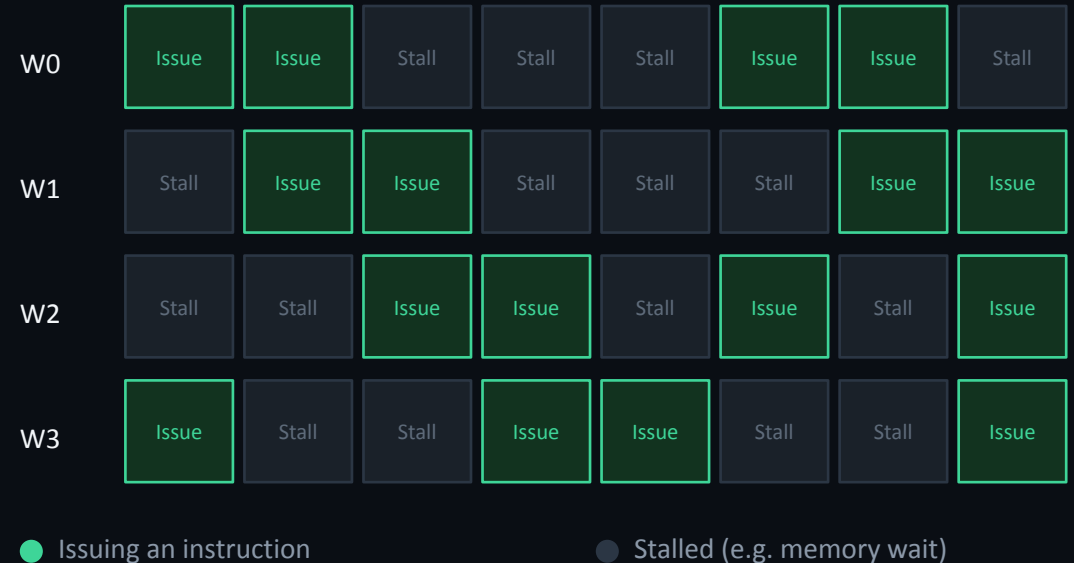


All lanes execute the same instruction each cycle — lockstep SIMT

Warp Scheduling: Hiding Latency

- Each SM partition holds many resident warps (e.g. up to 16), far more than can issue at once
- Every cycle, the scheduler picks a warp that is 'ready' (not stalled on memory/dependency) and issues its next instruction
- When a warp stalls (e.g. waiting on an HBM load), the scheduler instantly switches to another ready warp — zero-cost context switch
- This latency hiding is why GPUs favor many lightweight threads over few fast ones
- Occupancy = resident warps ÷ max warps per SM — higher occupancy gives the scheduler more ready warps to hide latency with

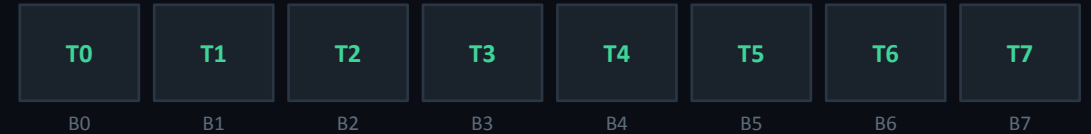
Scheduler Timeline (4 warps)



Shared Memory & Bank Conflicts

- Shared memory is a fast, on-chip, programmer-managed scratchpad shared by all threads in a block
- It's physically divided into 32 equal-width banks so up to 32 accesses can happen in parallel
- Consecutive 4-byte words map to consecutive banks (word $i \rightarrow \text{bank } i \bmod 32$)
- Conflict-free: each thread hits a different bank $\rightarrow$ single-cycle service
- Bank conflict: ≥ 2 threads hit the same bank with different addresses $\rightarrow$ access serializes, hurting throughput
- Broadcast (all threads read the identical address) is a special case — not a conflict

Conflict-Free Access (stride 1)



2-Way Bank Conflict (stride 4)



Threads 0–3 and 4–7 land on the same 4 banks $\rightarrow$ those accesses serialize into 2 phases

Fix: pad array rows (+1 element) or change access stride so threads map to distinct banks.

ALU Units: The CUDA Cores

- CUDA cores are the SM's scalar ALUs — one FP32 (and often INT32) FMA per core per cycle
- Grouped into partitions; each partition's warp scheduler issues to its own set of cores
- A warp instruction issues across 32 lanes — if a partition has 32 FP32 cores, a full warp completes in ~1 cycle
- Separate FP32, INT32 and sometimes FP64 pipelines allow some instruction mixing per cycle
- Handle general-purpose arithmetic: adds, multiplies, FMAs, comparisons, logic — the workhorses for non-matrix code



Each small square = one ALU (CUDA core) — warp lanes map 1:1 onto a partition's cores

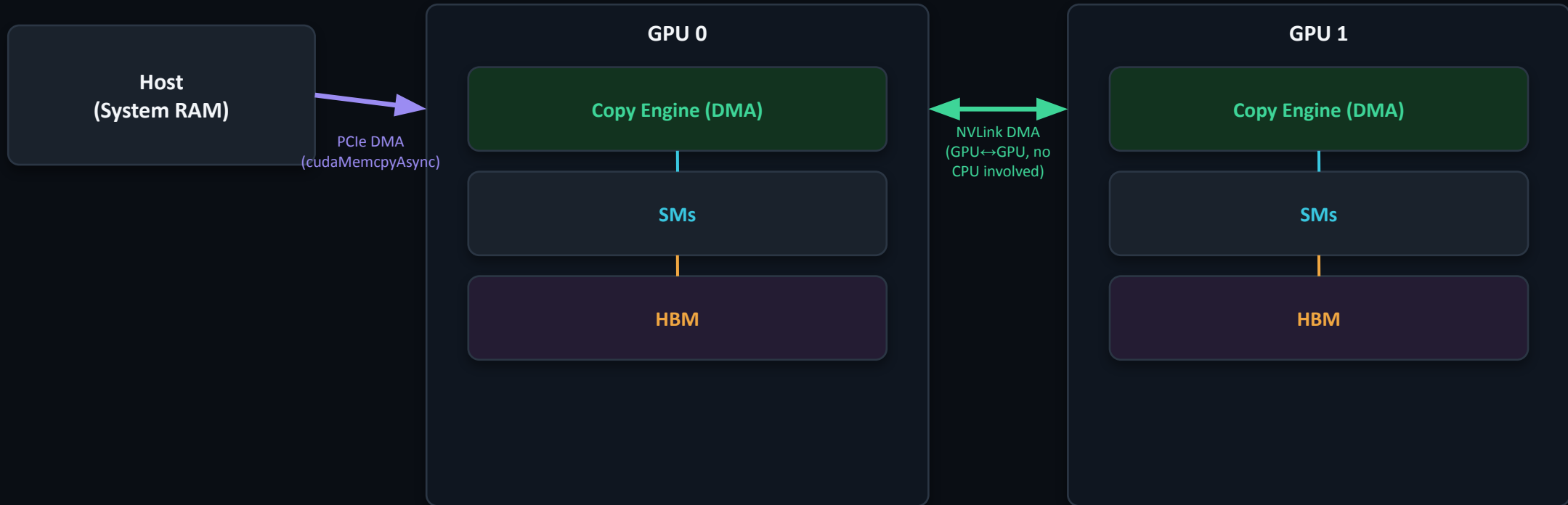
Tensor Cores: Matrix Engines

- Specialized units that compute small matrix multiply-accumulates in hardware: $D = A \times B + C$
- Operate on whole tiles (e.g. 16×16) per warp instruction, not one scalar per thread
- Support mixed precision — FP16/BF16/FP8/INT8 inputs with FP32 accumulation for stability
- Deliver an order of magnitude more FLOPs/cycle than CUDA cores for matmul-heavy workloads
- This is what dominates training/inference throughput for transformers — attention and MLP layers are almost all matmuls



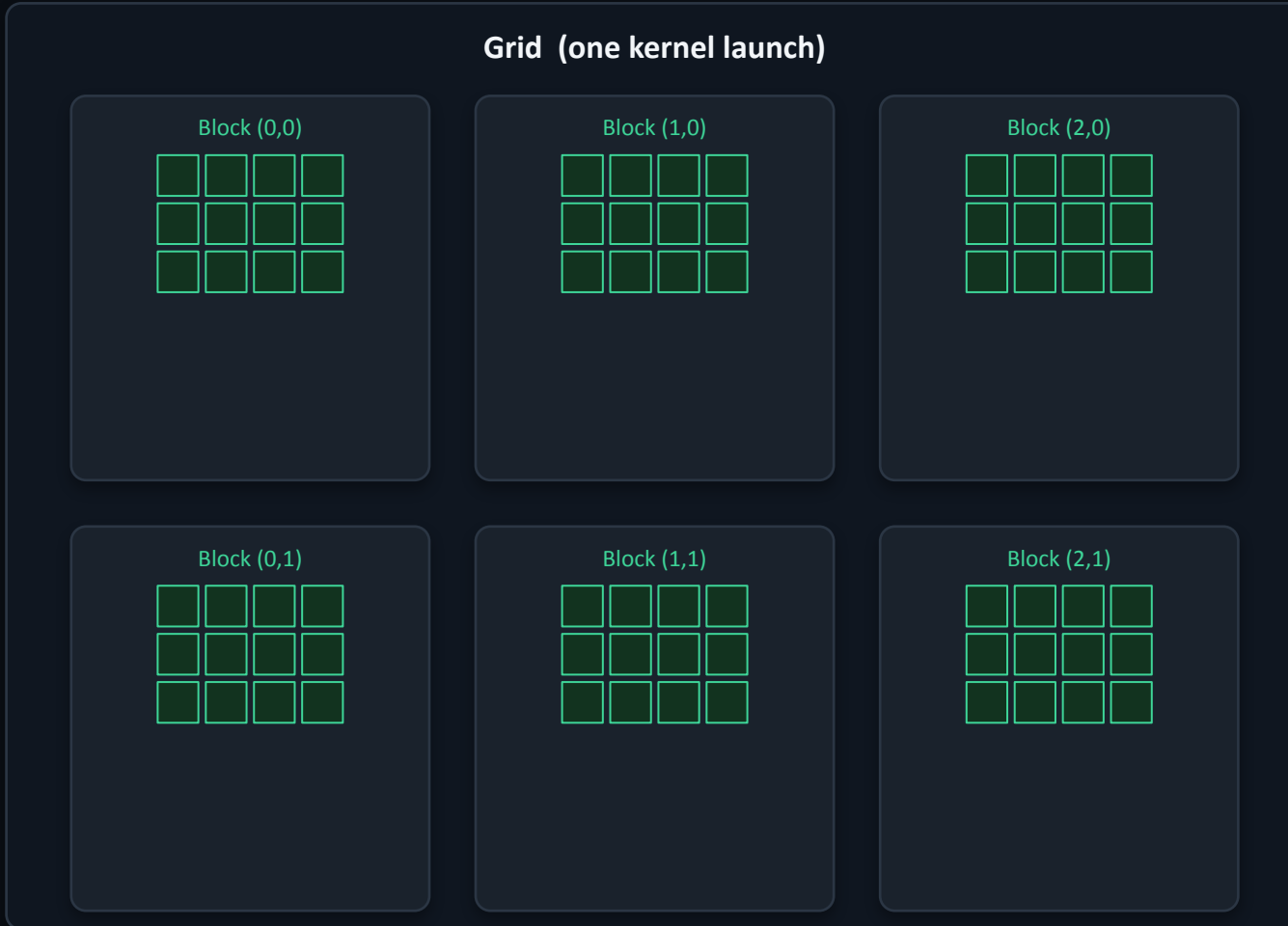
One tensor-core instruction $\equiv$ many CUDA-core cycles of scalar FMAs — the tile is computed as a single warp-wide operation.

Asynchronous Transfers via DMA Engines



- DMA (copy) engines move data independently of the SMs — compute and data movement overlap
- cudaMemcpyAsync + streams let a kernel on one stream run while another stream's transfer proceeds
- Peer-to-peer (P2P) NVLink copies go GPU HBM → GPU HBM directly, bypassing host memory entirely
- PCIe DMA still required for host ↔ device staging (input batches, results back to host)
- Pinned (page-locked) host memory is required for true async/DMA transfers — pageable memory forces a staged, synchronous copy

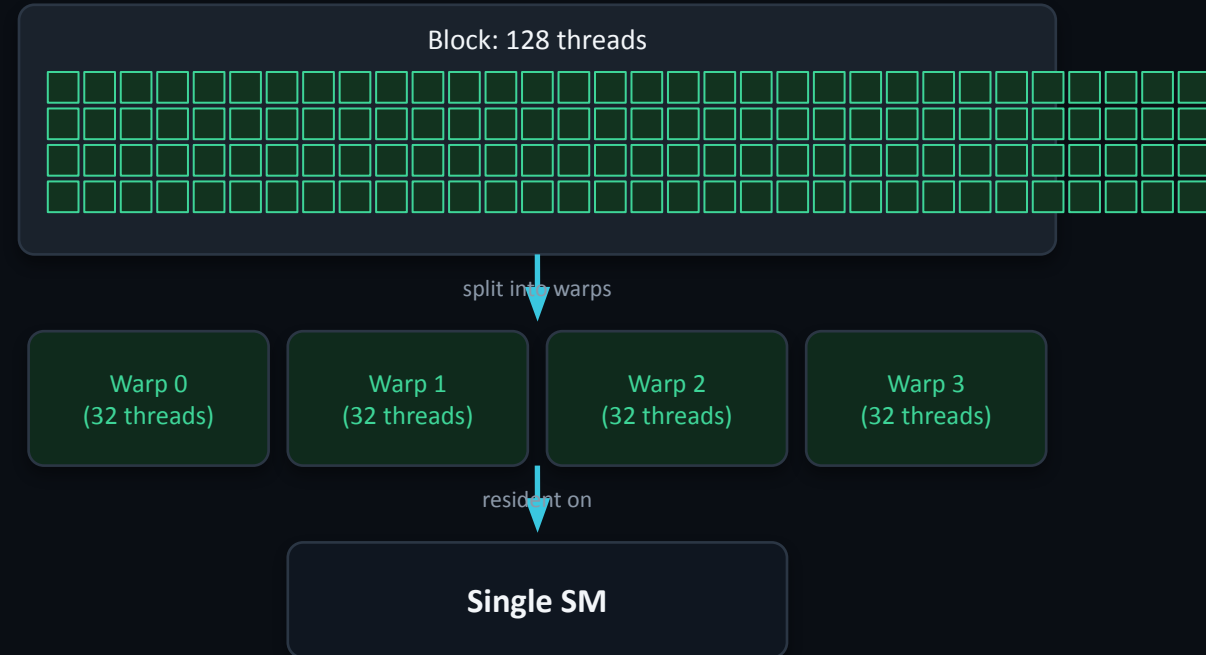
The Execution Hierarchy: Grid → Block → Thread



- Grid: all threads launched by one kernel call, organized as blocks
- Block: a group of threads (e.g. 256) that run on the same SM and can share memory / synchronize
- Thread: the individual execution unit; identified by threadIdx within its block and blockIdx within the grid
- Dimensions can be 1D, 2D or 3D — convenient for vectors, images, volumes
- Global thread index: $\text{idx} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$

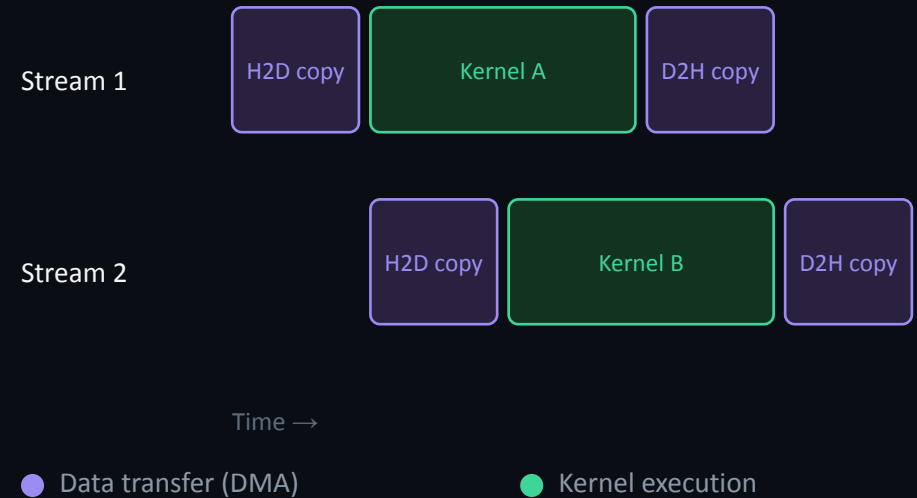
Threads in Blocks → Warps → SM

- A block is assigned, whole, to a single SM — it never spans multiple SMs
- The SM splits the block into warps of 32 consecutive threads for scheduling
- Multiple blocks can be resident on one SM simultaneously, limited by registers, shared memory, and warp slots
- Threads within a block can cooperate: shared memory + `__syncthreads()`
- Threads in different blocks cannot directly synchronize — only via separate kernel launches or grid-wide primitives



Streams: Overlapping Work

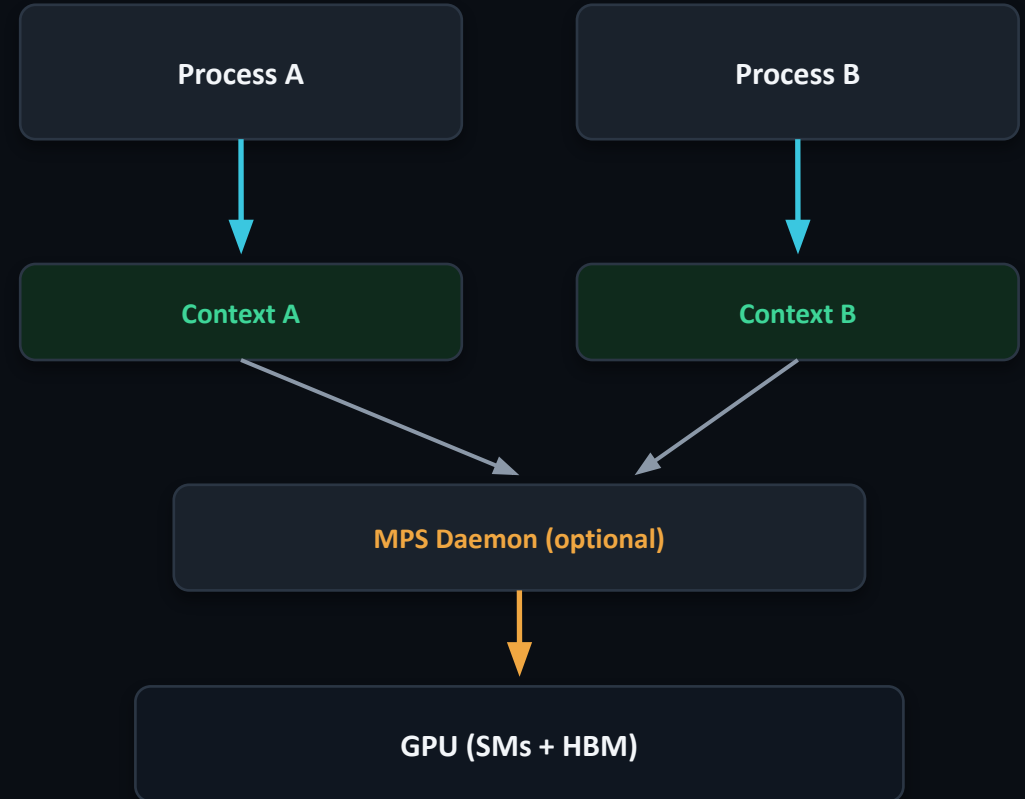
- A stream is an ordered queue of GPU operations (kernels, memcopies) issued by the host
- Operations within one stream execute in issue order
- Operations in different streams may run concurrently — kernel on stream A overlapping a copy on stream B
- The default (null) stream is implicitly synchronizing with other streams on legacy behavior — use explicit streams to get real concurrency
- Great for pipelining: overlap host→device transfer of chunk N+1 with compute on chunk N



Stream 2's transfer overlaps Stream 1's kernel — the copy engine and SMs work concurrently.

Process Context: Who Owns the GPU State

- A CUDA context is analogous to a CPU process's address space — it owns allocated memory, loaded modules, and streams on a device
- Created implicitly on first CUDA call, or explicitly managed with the driver API
- Each host process gets its own context per GPU by default — contexts isolate state between processes
- Switching context on a GPU (multiple processes time-sharing one GPU) has overhead



Synchronization: Keeping Threads in Step



Within a Block

- `__syncthreads()` — a barrier: every thread in the block must arrive before any proceeds
- Used to make shared-memory writes visible before other threads read them
- Must be reached uniformly — placing it inside divergent conditional code is undefined behavior
- `__syncwarp()` offers a lighter-weight barrier for just the 32 threads in a warp



Across the Grid / Host

- `cudaDeviceSynchronize()` — host waits until all prior work on the device completes
- `cudaStreamSynchronize(stream)` — host waits for one specific stream
- CUDA events (`cudaEventRecord/Synchronize`) let streams wait on each other without blocking the host
- Cooperative groups' `grid.sync()` allows a true grid-wide barrier for kernels launched cooperatively

Rule of thumb: sync only as widely as you must — warp → block → stream → device, in order of increasing cost.

SUMMARY

Putting It All Together



System: CPUs + GPUs linked by PCIe; GPUs mesh via NVLink/NVSwitch for scale-up



Chip: SM array does the compute; HBM stacks deliver the bandwidth



SM: Threads group into warps; schedulers hide latency; ALUs + tensor cores do the math



Memory: Shared memory banks give parallel on-chip access; DMA engines move data asynchronously



CUDA model: Grid → block → thread hierarchy, streams for overlap, contexts for isolation, barriers for correctness

The throughline: GPUs win by exposing massive, structured parallelism — across chips, across SMs, and across the 32 lanes of every warp — while hardware schedulers and async DMA keep that parallelism fed.



PART TWO

Writing CUDA Kernels

From grid/block/thread setup to shared-memory tiling and tensor-core MMA

What this section adds: concrete, compilable CUDA C++ — how to launch a grid of blocks and threads, allocate and move memory, and three real kernels: a basic vector add, a shared-memory tiled matmul, and a tensor-core WMMA matmul, tying back to the SM/memory concepts from earlier in this deck.

Defining Grids, Blocks & Threads in Code

Kernel is divided into a grid of thread blocks. Each block is also a grid of threads.

```
dim3 threadsPerBlock(256);           // 1D block of 256 threads
dim3 blocksPerGrid((N + 255) / 256); // enough blocks to cover all N elements
myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

// 2D example – common for image/matrix work, e.g. 16×16 threads per block:
dim3 threads2D(16, 16);
dim3 blocks2D((width + 15) / 16, (height + 15) / 16);
kernel2D<<<blocks2D, threads2D>>>(...);
```

```
__global__ void myKernel(const float* A, const float* B, float* C, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x; // this thread's GLOBAL index
    if (idx < N) { /* ... */ }                       // bounds check – grid is often larger than N
}
```

The four built-ins every kernel uses: `gridDim` (blocks in the grid), `blockDim` (threads in a block), `blockIdx` (this block's position in the grid), `threadIdx` (this thread's position in its block). Global index = `blockIdx × blockDim + threadIdx` — the same formula from the CUDA basics section earlier, now in real code. `<<<grid, block>>>` is the launch syntax; both can be a plain `int` (1D) or a `dim3` (2D/3D).

Allocating GPU Memory & Moving Data

The standard explicit pattern:

```
float *h_A, *d_A;
size_t bytes = N * sizeof(float);

h_A = (float*)malloc(bytes);           // host allocation (pageable)
cudaMalloc(&d_A, bytes);               // device allocation — lives in GPU HBM

cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice); // H2D copy
/* ... launch kernel(s) that read/write d_A ... */
cudaMemcpy(h_A, d_A, bytes, cudaMemcpyDeviceToHost); // D2H copy back

cudaFree(d_A);   free(h_A);
```

Alternative: Unified Memory (one pointer, accessible from both sides):

```
float *A;
cudaMallocManaged(&A, bytes); // driver migrates pages between host/device automatically on access
```

Recall from earlier in this deck: plain `malloc()` gives PAGEABLE host memory — `cudaMemcpyAsync` on it silently becomes a staged, synchronous copy. For real async H2D/D2H overlap, allocate host memory with `cudaMallocHost()` (pinned) instead of `malloc()`.

Your First Kernel: Vector Add

```
__global__ void vecAdd(const float* A, const float* B, float* C, int N) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    if (idx < N) {  
        C[idx] = A[idx] + B[idx];  
    }  
}  
  
int threads = 256;  
int blocks = (N + threads - 1) / threads;  
vecAdd<<<blocks, threads>>>>(d_A, d_B, d_C, N);  
cudaDeviceSynchronize();           // kernel launches are ASYNC — wait before reading results
```

Grid-stride loop — for when N is bigger than the total threads you launched:

```
__global__ void vecAddStrided(const float* A, const float* B, float* C, int N) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    int stride = blockDim.x * gridDim.x;           // total threads launched  
    for (int i = idx; i < N; i += stride)           // each thread handles multiple elements  
        C[i] = A[i] + B[i];  
}
```

Why grid-stride matters: you don't have to launch exactly N threads. Launch a grid sized for the GPU's actual SM count, and let each thread loop over several elements — avoids re-launch overhead and works for any N without recomputing grid dimensions.

A Shared-Memory Kernel: Tiled Matrix Multiply

Data in Shared Memory is available to the whole block.

```
#define TILE 16
__global__ void matMulTiled(float* A, float* B, float* C, int N) {
    __shared__ float As[TILE][TILE];           // on-chip, shared by every thread in this block
    __shared__ float Bs[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < N / TILE; ++t) {
        As[threadIdx.y][threadIdx.x] = A[row * N + t * TILE + threadIdx.x];
        Bs[threadIdx.y][threadIdx.x] = B[(t * TILE + threadIdx.y) * N + col];
        __syncthreads();                       // wait until the WHOLE tile has loaded

        for (int k = 0; k < TILE; ++k)
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x]; // reused TILE times from fast on-chip memory
        __syncthreads();                       // wait before the next iteration overwrites the tile
    }
    C[row * N + col] = sum;
}
```

Why this matters: each element loaded from HBM is reused TILE times from on-chip shared memory instead of re-read from HBM every time — raising arithmetic intensity and moving this kernel toward the compute-bound side of the roofline from earlier in this deck. `__syncthreads()` is required here for correctness: without it, a thread could read a tile element before another thread finished writing it.

A Tensor-Core Kernel: the WMMA API

```
#include <mma.h>
using namespace nv_cuda;

__global__ void wmmaMatMul(half* A, half* B, float* C, int M, int N, int K) {
    // One WARP (32 threads) cooperatively computes one 16x16 output tile
    wmma::fragment<wmma::matrix_a, 16,16,16, half, wmma::row_major>    a_frag;
    wmma::fragment<wmma::matrix_b, 16,16,16, half, wmma::row_major>    b_frag;
    wmma::fragment<wmma::accumulator, 16,16,16, float>                c_frag;
    wmma::fill_fragment(c_frag, 0.0f);

    int warpM = blockIdx.y * blockDim.y + threadIdx.y;
    int warpN = (blockIdx.x * blockDim.x + threadIdx.x) / warpSize;

    for (int k = 0; k < K; k += 16) {
        wmma::load_matrix_sync(a_frag, A + warpM*16*K + k, K);
        wmma::load_matrix_sync(b_frag, B + k*N + warpN*16, N);
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);    // the actual tensor-core instruction
    }
    wmma::store_matrix_sync(C + warpM*16*N + warpN*16, c_frag, N, wmma::mem_row_major);
}
```

Mapping back to the SM internals section: a fragment is the on-chip register tile the tensor core reads/writes; mma_sync is literally the $D = A \times B + C$ tensor-core instruction from earlier, issued once per warp per $16 \times 16 \times 16$ chunk. In practice, cuBLAS / CUTLASS / Triton wrap this API — hand-written WMMA is for custom fused kernels.

Putting It Together, and Common Pitfalls

```
int main() {
    int N = 1 << 20;
    size_t bytes = N * sizeof(float);
    float *h_A, *h_B, *h_C;          // allocate + init on host
    float *d_A, *d_B, *d_C;

    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);

    cudaMemcpy(d_A, h_A, bytes,
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes,
               cudaMemcpyHostToDevice);

    int t = 256, b = (N+t-1)/t;
    vecAdd<<<b, t>>>>(d_A, d_B, d_C, N);

    cudaMemcpy(h_C, d_C, bytes,
               cudaMemcpyDeviceToHost);
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
```



Missing the if (idx < N) bounds check — out-of-bounds reads/writes when grid size doesn't divide evenly



Forgetting cudaDeviceSynchronize() — kernel launches return immediately; reading results too early races the kernel



Not checking cudaGetLastError() — a bad launch config fails silently otherwise



Using plain malloc() where you need real async transfer overlap — needs cudaMallocHost() (pinned)



Missing __syncthreads() after a shared-memory write and before another thread reads it



Shared-memory bank conflicts from a bad access stride (see the SM-internals section earlier)



PART THREE

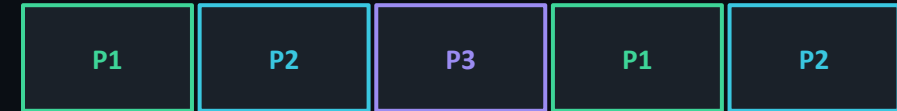
GPU Throughput

Setup · Verification · Measured Performance Gains · SM/Thread Utilization Analysis

GPU Throughput Problems

- Every process that touches a GPU gets its OWN CUDA context
- The GPU/driver can only actively run ONE context's work at a time — contexts are TIME-SLICED
- Switching between contexts has real overhead (context-switch cost)
- Consequence: N processes sharing a GPU take turns — switch overhead eats into useful compute time
- Even if one process's kernel doesn't use the whole GPU, that idle space stays unused — no OTHER context's kernel can co-run in the gap

Time-Sliced Execution (No MPS)



What Is NVIDIA MPS?

- MPS = CUDA Multi-Process Service — a background daemon (nvidia-cuda-mps-control + nvidia-cuda-mps-server)
- Lets multiple OS processes share ONE GPU through a SINGLE shared CUDA context, instead of each process getting its own separate context
- Ships as part of the NVIDIA driver install — nvidia-smi, nvidia-cuda-mps-control, nvidia-cuda-mps-server are already present
- No extra package installation is required to use MPS



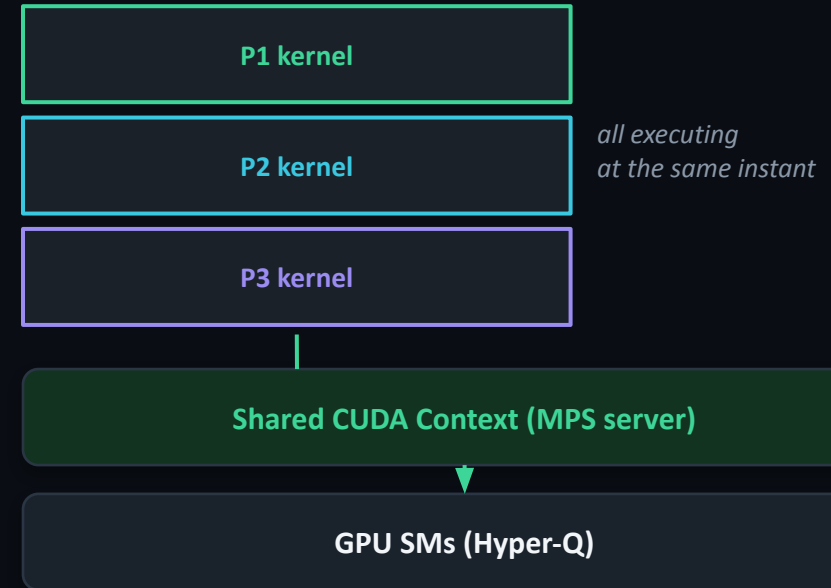
The Two Daemons

- nvidia-cuda-mps-control — the control daemon; started once, accepts admin commands
- nvidia-cuda-mps-server — spawned automatically on first client connect; the ONE shared context owner

What MPS Actually Changes (With MPS)

- All client processes funnel their kernel launches through ONE shared context, owned by nvidia-cuda-mps-server
- No context to switch between — kernels from DIFFERENT processes become entries in the SAME hardware work queues (Hyper-Q)
- They can be scheduled onto the GPU's SMs CONCURRENTLY (spatial sharing) instead of taking turns

Concurrent Execution (With MPS)



When Does MPS Actually Help?

The one-sentence mental model

MPS helps exactly when a SINGLE process's kernels do NOT use all of the GPU's SMs by themselves — the idle SMs left over become available for OTHER processes' kernels to fill concurrently.

- If a kernel already uses ALL the SMs (e.g. big training-batch GEMMs), there's no idle capacity for MPS to give away
- MPS still helps a little in that case (removes context-switch overhead), but not via genuine concurrency
- This is exactly why the SM/occupancy diagnostic (covered later in this section) is the “before you even try MPS” step

Rule of thumb: underused SMs = MPS opportunity. Saturated SMs = MPS gives context-switch removal only.

The 3 GPU Compute Modes That Matter

Mode	Behavior
Default	Multiple processes/contexts may use the GPU directly (each gets time-sliced access). Factory default.
Exclusive_Process	Only ONE process may hold a CUDA context on the GPU at a time. RECOMMENDED for MPS — forces every other would-be direct client to fail to grab its own context, guaranteeing the MPS server is the sole context holder.
Exclusive_Thread (legacy)	Deprecated/legacy
Prohibited	No process may use the GPU. Not relevant to MPS.

Why Exclusive_Process for MPS: it structurally guarantees every real client is forced through the MPS server — no process can accidentally bypass MPS and grab its own direct context on that GPU.

Checking, Setting & Compute-Mode Gotchas

```
# Check current mode
nvidia-smi --query-gpu=index,compute_mode \
  --format=csv -i 0

# Set to Exclusive_Process (for MPS)
nvidia-smi -i 0 -c EXCLUSIVE_PROCESS

# Set back to Default
nvidia-smi -i 0 -c DEFAULT
```

- Changing compute mode requires NO active CUDA processes on that GPU — check first with `--query-compute-apps`
- Compute mode is HOST-WIDE, not scoped to your session — changing it affects any other user/process on that GPU. Only do this on a confirmed-idle GPU.



Gotcha hit repeatedly: MPS server holds the context slot even when idle

Once an MPS server has ever accepted a client on an Exclusive_Process GPU, that server process continues to hold the one allowed context slot — even with zero active clients. A separate NON-MPS process (e.g. `ncu` or `nsys` run directly) then fails with:

```
"CUDA error: CUDA-capable device(s) is/are busy or unavailable"
```

Fix used every time: switch the GPU back to DEFAULT mode before running `ncu/nsys` directly, then switch back to EXCLUSIVE_PROCESS afterward.

Step-by-Step MPS Setup for GPU 0

1 Confirm GPU is idle

```
nvidia-smi -i 0 --query-compute-apps=pid --format=csv,noheader (should print nothing)
```

2 Set compute mode

```
CUDA_VISIBLE_DEVICES=0 nvidia-smi -i 0 -c EXCLUSIVE_PROCESS
```

3 Start the control daemon

```
CUDA_VISIBLE_DEVICES=0 CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_gpu0 \  
CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log0 nvidia-cuda-mps-control -d
```

4 Verify the daemon is alive

```
pid=$(cat /tmp/mps_gpu0/nvidia-cuda-mps-control.pid); echo "$pid"; ps -p "$pid" -o pid,cmd
```

5 Verify it responds

```
echo "get_server_list" | CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_gpu0 nvidia-cuda-mps-control → empty  
(healthy, no clients yet)
```

Client Connections & the Remapped-Index Gotcha

How client processes connect:

```
export CUDA_VISIBLE_DEVICES=0
export CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_gpu0
export CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log0
<run your CUDA/PyTorch/vLLM program normally>
```

- If `CUDA_MPS_PIPE_DIRECTORY` is unset (or points at a dead daemon), the client silently falls back to non-MPS behavior — always verify, don't assume



Gotcha: remapped device index

Tearing down MPS:

```
echo quit | CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_gpu0 \
nvidia-cuda-mps-control
nvidia-smi -i 0 -c DEFAULT
```

“It Didn't Error Out” Is Not Proof

The standard: aggregate throughput going up is suggestive but NOT direct proof of concurrent kernel execution — it could theoretically be explained by other factors. This investigation insisted on DIRECT evidence: GPU kernel start/end timestamps checked for temporal overlap across different OS processes.

Quick qualitative check: control / server logs

- While a benchmark with N processes runs through MPS, the control log shows ALL clients being served by ONE shared server (“Server already exists” for the 2nd, 3rd... client)
- Confirm exactly one nvidia-cuda-mps-server process exists, parented by the control daemon, with N independent client PIDs attached

```
tail /tmp/mps_log0/control.log
-> [...] NEW CLIENT <pid> from user 0: Server already exists

ps -ef | grep -i mps
-> nvidia-cuda-mps-control -d      (the control daemon)
-> nvidia-cuda-mps-server          (ONE shared server, spawned on first client connect)
```

Rigorous Proof: nsys Timeline + Sweep-Line

- 1. Run the multi-process workload under nsys profile, tracing CUDA, with --trace-fork-before-exec=true (required — Python multiprocessing ‘spawn’ does fork+exec; without this flag nsys misses the child processes entirely)
- 2. Repeat WITHOUT MPS (compute mode Default, no MPS env vars) for contrast
- 3. Export each trace to SQLite for direct querying
- 4. Query CUPTI_ACTIVITY_KIND_KERNEL for (start, end, globalPid) of the GEMM kernel, then run a SWEEP-LINE algorithm: sort by start time, track currently-active intervals, flag when a new launch overlaps an interval from a DIFFERENT process

```
env CUDA_VISIBLE_DEVICES=0 CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_gpu0 CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log0 \  
  nsys profile --trace=cuda --trace-fork-before-exec=true --force-overwrite=true \  
  -o with_mps_trace python3 mps_benchmark.py --gpu 0 --num-procs 3 --duration 3 --matrix-size 256  
  
nsys export --type sqlite --force-overwrite=true -o with_mps_trace.sqlite with_mps_trace.nsys-rep  
  
# Then: sort kernel launches by start time; for each new launch, check whether any  
# currently-active interval belongs to a DIFFERENT globalPid (sweep-line overlap check)  
  
python3 sweep\ line\ sqlite.py with_mps_trace.sqlite
```

Proof Results: 0% → 20.7% Kernel Overlap

Metric	WITHOUT MPS	WITH MPS
GEMM launches captured (3 procs)	220,712	262,818
Launches overlapping a different process's kernel	0 (0.00%)	54,440 (20.7%)
Concurrent process pairs observed	NONE	ALL 3 possible pairs
Max concurrent processes with an active kernel	1	3
Total overlapping GPU-time (3s trace window)	—	116.5 ms



Takeaway: at times, ALL THREE processes had a GEMM kernel executing on the GPU AT THE SAME INSTANT. Direct, measured proof of concurrent kernel execution — not inferred from throughput, but read directly off the GPU's own kernel timestamps. Without MPS: fully serialized, proving the “time-sliced, one at a time” model.

Methodology: The A/B Test Pattern

```
# WITHOUT MPS
```

```
nvidia-smi -i 0 -c DEFAULT;
```

```
env CUDA_VISIBLE_DEVICES=0 nsys profile --trace=cuda
--trace-fork-before-exec=true --force-overwrite=true
-o without_mps_trace python3 mps_benchmark.py --gpu 0
--num-procs 3 --duration 15 --matrix-size 256;
```

```
nsys export --type sqlite --force-overwrite=true -o
without_mps_trace.sqlite without_mps_trace.nsys-rep;
```

```
python3 sweep\ line\ sqlite.py
without_mps_trace.sqlite
```

```
# WITH MPS
```

```
nvidia-smi -i 0 -c EXCLUSIVE_PROCESS;
```

```
env CUDA_VISIBLE_DEVICES=0
CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_gpu0
CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log0 nsys profile
--trace=cuda --trace-fork-before-exec=true
--force-overwrite=true -o with_mps_trace python3
mps_benchmark.py --gpu 0 --num-procs 3 --duration 15
--matrix-size 256;
```

```
nsys export --type sqlite --force-overwrite=true -o
with_mps_trace.sqlite with_mps_trace.nsys-rep;
```

```
python3 sweep\ line\ sqlite.py with_mps_trace.sqlite
```

This exact pattern repeats across synthetic matmul, raw HF transformers Llama, and real vLLM — only the workload script changes.

Results: Synthetic FP16 Matmul

Workload	Without MPS	With MPS	Gain
4096×4096 matmul (6 procs)	1,812.23 iters/s	2,103.80 iters/s	+16% (1.16x)
256×256 matmul (6 procs)	56,262.74 iters/s	165,362.01 iters/s	+194% (2.94x)



Why the difference: fully explained by SM occupancy (Section 11). The 4096 case already uses ALL 132 SMs per kernel — no spare capacity. The 256 case only uses 64 of 132 SMs per kernel — plenty of spare capacity for other processes to fill concurrently. Same GPU, same MPS setup, wildly different gain — because the underlying occupancy is different.

Results: Real Llama-3.1-8B (Raw HF Transformers)

Batch Size	Without MPS (tok/s)	With MPS (tok/s)	Speedup
1	56.17	92.93	1.65x
16	767.31	1,325.31	1.73x
32	1,441.13	2,350.32	1.63x

Consistent ~1.6–1.7x speedup across batch sizes — real model, real generation loop, not a synthetic proxy. This is the number a raw HuggingFace-transformers serving setup would actually see from enabling MPS across 3 co-located replicas.

Results: Real vLLM 0.3.2 + Llama-3-8B

Batch Size	Without MPS (tok/s)	With MPS (tok/s)	Speedup
1	105.82	187.06	1.77x
16	1,656.70	2,784.81	1.68x
32	3,113.53	4,981.61	1.60x

vLLM's own kernels (continuous batching, paged attention, fused kernels) make each PROCESS individually faster than raw HF transformers (e.g. 120.9 vs 44.1 tok/s single-process at batch=1), but do NOT eliminate the SM-underutilization gap — MPS still delivers a comparable, real gain on top of vLLM's own optimizations.

Overhead / Cost of MPS (Measured, Not Assumed)

Latency overhead

Single process (no contention) — isolates the pure cost of routing kernel launches through the MPS client→server IPC hop:

Run	iters/sec (avg of 3)
Without MPS	37,987.61
With MPS	38,940.42

Delta: +2.51% — WITHIN run-to-run noise (ranges overlap). No measurable per-launch latency overhead at this granularity.

Memory overhead

```
nvidia-smi -i 0 --query-compute-apps=\
    pid,process_name,used_memory --format=csv
-> nvidia-cuda-mps-server, 58 MiB
```

Under MPS, memory allocated by CLIENTS is attributed to the shared SERVER process (it owns the real context). 58 MiB is the shared context's bookkeeping overhead (cuBLAS handles, driver workspace) — ~0.07% of an 80GB H100, negligible, but a PER-SERVER cost, not per-client.

Bottom line: for this workload granularity, MPS's IPC hop (documented at single-digit microseconds) is lost in the noise of tens-of-microseconds kernel + Python dispatch time. Could matter more for extremely tiny/rapid-fire kernels with no Python overhead in between.

The Real Cost of MPS: Reliability, Not Performance



Extra long-lived processes

nvidia-cuda-mps-control and nvidia-cuda-mps-server — trivial CPU/RAM cost, but must be remembered and torn down.



Fault isolation loss

All connected clients share ONE underlying CUDA context. A client that triggers a GPU exception (illegal memory access, etc.) can bring down the SHARED SERVER — disrupting or terminating ALL other currently-connected clients. Without MPS, a crash in one process's own context does NOT directly affect others.



Shared resource cap

All clients share the server's default 100% “active thread percentage” quota and free-for-all compete for SMs, unless explicitly partitioned via `set_active_thread_percentage` (per-client) or `set_default_active_thread_percentage` (daemon-wide).

This is the main real tradeoff of MPS — a reliability/blast-radius cost, not a speed cost.

Kernel Preemption

```
# WITHOUT MPS
```

```
nvidia-smi -i 0 -c DEFAULT;
```

```
env CUDA_VISIBLE_DEVICES=0 nsys profile --trace=cuda
--trace-fork-before-exec=true --force-overwrite=true
-o without_mps_trace python3 mps_benchmark.py --gpu 0
--num-procs 3 --duration 15 --matrix-size 4096;
```

```
nsys export --type sqlite --force-overwrite=true -o
without_mps_trace.sqlite without_mps_trace.nsys-rep;
```

```
python3 sweep\ line\ sqlite.py
without_mps_trace.sqlite
```

```
# WITH MPS
```

```
nvidia-smi -i 0 -c EXCLUSIVE_PROCESS;
```

```
env CUDA_VISIBLE_DEVICES=0
CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_gpu0
CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log0 nsys profile
--trace=cuda --trace-fork-before-exec=true
--force-overwrite=true -o with_mps_trace python3
mps_benchmark.py --gpu 0 --num-procs 3 --duration 15
--matrix-size 4096;
```

```
nsys export --type sqlite --force-overwrite=true -o
with_mps_trace.sqlite with_mps_trace.nsys-rep;
```

```
python3 sweep\ line\ sqlite.py with_mps_trace.sqlite
```

This exact pattern repeats across synthetic matmul, raw HF transformers Llama, and real vLLM — only the workload script changes.

The Core Formula & the ncu Occupancy Tool

What determines a GEMM kernel's grid size

$$\text{grid_size (thread blocks)} = \text{ceil}(M / \text{tile_M}) * \text{ceil}(N / \text{tile_N})$$

tile_M/tile_N is a hardware- and shape-dependent choice made by cuBLAS's kernel-selection heuristic — not set directly in plain PyTorch/cuBLAS calls. Each thread block is normally scheduled to a DIFFERENT SM, so if grid_size < total_SMs, some SMs are PROVABLY left idle.

ncu (Nsight Compute) Occupancy section

```
nvidia-smi -i 0 -c DEFAULT; env CUDA_VISIBLE_DEVICES=0 ncu --target-processes all --section Occupancy --kernel-name "regex:gemm" --launch-count 1 --launch-skip 10 python3 mps_benchmark.py --gpu 0 --num-procs 3 --duration 15 --matrix-size 4096;
```

Metric	Meaning
Grid Size	total thread blocks launched
Block Limit SM / Registers / Shared Mem / Warps	max resident blocks/SM under each constraint
Achieved Occupancy	measured warp-slot utilization, %

The BINDING CONSTRAINT is whichever “Block Limit” number is smallest — usually Registers or Shared Mem, NOT Warps or SM.

sm90_xmma_gemm_f16f16_f16f32_f32_nn_n_tilesize128x128x64_warpgroupsize1x1x1_execute_segment_k_of
f_kernel_51_cublas (132, 1, 1)x(384, 1, 1), Context 1, Stream 7, Device 0, CC 9.0

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Max Active Clusters	cluster	66
Max Cluster Size	block	16
Overall GPU Occupancy	%	0.59
Cluster Occupancy	%	3.12
Block Limit SM	block	32
Block Limit Registers	block	1
Block Limit Shared Mem	block	1
Block Limit Warps	block	5
Theoretical Active Warps per SM	warp	12
Theoretical Occupancy	%	18.75
Achieved Occupancy	%	16.62
Achieved Active Warps Per SM	warp	10.64

WRN This kernel's theoretical occupancy (18.8%) is limited by the number of required registers. This kernel's
theoretical occupancy (18.8%) is limited by the required amount of shared memory. See the CUDA Best

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Max Active Clusters	cluster	0
Max Cluster Size	block	8
Overall GPU Occupancy	%	0
Cluster Occupancy	%	0
Block Limit SM	block	32
Block Limit Registers	block	8
Block Limit Shared Mem	block	4
Block Limit Warps	block	16
Theoretical Active Warps per SM	warp	16
Theoretical Occupancy	%	25
Achieved Occupancy	%	6.25
Achieved Active Warps Per SM	warp	4.00

Grid $\geq$ SM Count $\neq$ Fully Packed

Early (wrong) assumption: “blocks_launched / total_SMs” as a proxy for “% of GPU used.” This only holds in ONE direction — grid_size < SM_count reliably means idle SMs; grid_size $\geq$ SM_count does NOT mean fully packed.

Measured: 4096×4096 fp16 GEMM

grid = 132 = SM count exactly

Metric	Value
Block Limit Registers	1
Block Limit Shared Mem	1
Achieved Occupancy	16.62%

Looks spare (16.62%) but ISN'T — registers+shared-mem cap this SM at exactly 1 resident block. Zero room for any second block. Why this case only got +16% from MPS.

Contrast: 256×256 fp16 GEMM

grid = 64 / 132 SMs

Metric	Value
Block Limit Shared Mem	4 blocks/SM
Achieved Occupancy	6.25%

REAL headroom two ways: 68 SMs get ZERO blocks, AND up to 3 more block-slots on the 64 SMs that do. This is the exploitable headroom MPS used to get 2.94x.

Correction #2: One Sample Can Mislead

- The SAME kernel PTX binary (e.g. “tilesize64x64x64”) is often reused by cuBLAS for SEVERAL DIFFERENT weight matrices in a model, each with a different N — hence different grid sizes for the SAME kernel name
- A single --launch-skip N --launch-count 1 sample can land on any of them almost at random and give a misleading number
- Fix: query the FULL distribution of grid sizes across ALL launches, from an nsys trace exported to SQLite

```
select k.gridX, count(*) from CUPTI_ACTIVITY_KIND_KERNEL k
join StringIds s on k.demangledName = s.id
where s.value like '%tilesize64x64x64%' group by k.gridX order by count(*) desc;
```

Example of the problem this caught (raw HF Llama-3.1-8B decode, batch=16):

gridX	count
64	1,824
120	1,216
63	64

Two distinct, consistently-recurring call sites share this one kernel: ~64 (48.5% of SMs) and ~120 (90.9% of SMs). A single sample would have reported only ONE of these.

```
> sqllite with_mps_trace.sqlite
```

```
> select gridX, gridY, gridZ, blockX, blockY, blockZ, staticSharedmemory,  
dynamicSharedMemory from CUPTI_ACTIVITY_KIND_KERNEL k JOIN StringIds s ON  
k.demangledName = s.id WHERE s.value LIKE '%tilesize128%' limit 10;
```

```
132|1|1|384|1|1|0|231424
```

```
132|1|1|384|1|1|0|231424
```

number of blocks - $\text{gridX} * \text{gridY} * \text{gridZ}$

Theads per block - $\text{blockX} * \text{blockY} * \text{blockZ}$

SharedMemory - $\text{staticSharedmemory} + \text{dynamicSharedMemory}$

Does DRAM Bandwidth Back This Up?

Profiled the same dominant decode GEMM kernel (grid=64/132 SMs):

```
nvidia-smi -i 0 -c DEFAULT;
```

```
env CUDA_VISIBLE_DEVICES=0 ncu --target-processes all --metrics  
dram__bytes_read.sum,dram__bytes_write.sum,gpu__time_duration.sum --section  
MemoryWorkloadAnalysis --kernel-name "regex:gemm" --launch-count 1 --launch-skip 10  
python3 mps_benchmark.py --gpu 0 --num-procs 3 --duration 15 --matrix-size 4096;
```

sm90_xmma_gemm_f16f16_f16f32_f32_nn_n_tilesize128x128x64_warpgroupsize1x1x1_execute_segment_k_of
f_kernel__51_cublas (132, 1, 1)x(384, 1, 1), Context 1, Stream 7, Device 0, CC 9.0

Section: Command line profiler metrics

Metric Name	Metric Unit	Metric Value
dram__bytes_read.sum	Mbyte	312.73
dram__bytes_write.sum	Mbyte	32.14
gpu__time_duration.sum	usecond	227.62

Section: Memory Workload Analysis

Metric Name	Metric Unit	Metric Value
Memory Throughput	Tbyte/second	1.52
Mem Busy	%	68.77
Max Bandwidth	%	67.25
L1/TEX Hit Rate	%	0
L2 Hit Rate	%	61.43
Mem Pipes Busy	%	50.42

sm80_xmma_gemm_f16f16_f16f32_f32_nn_n_tilsize32x32x64_stage6_warpsize2x2x1_tensor16x8x16_execute_kernel__51_cublas (8, 8, 1)x(128, 1, 1), Context 1, Stream 7, Device 0, CC 9.0

Section: Command line profiler metrics

Metric Name	Metric Unit	Metric Value
dram_bytes_read.sum	Kbyte	281.09
dram_bytes_write.sum	byte	0
gpu_time_duration.sum	usecond	5.41

Section: Memory Workload Analysis

Metric Name	Metric Unit	Metric Value
Memory Throughput	Gbyte/second	51.98
Mem Busy	%	9.21
Max Bandwidth	%	5.93
L1/TEX Hit Rate	%	0
L2 Hit Rate	%	90.38
Mem Pipes Busy	%	2.69

SpeedOfLight Section

Profiled the same dominant decode GEMM kernel (grid=64/132 SMs):

```
nvidia-smi -i 0 -c DEFAULT;  
  
env CUDA_VISIBLE_DEVICES=0 ncu --target-processes all --metrics  
dram__bytes_read.sum,dram__bytes_write.sum,gpu__time_duration.sum --section  
MemoryWorkloadAnalysis --section SpeedOfLight --kernel-name "regex:gemm" --launch-count 1  
--launch-skip 10 python3 mps_benchmark.py --gpu 0 --num-procs 3 --duration 15  
--matrix-size 4096;
```


Section: GPU Speed Of Light Throughput

Metric Name	Metric Unit	Metric Value
DRAM Frequency	cycle/nsecond	2.62
SM Frequency	cycle/nsecond	1.25
Elapsed Cycles	cycle	288748
Memory Throughput	%	68.53
DRAM Throughput	%	44.51
Duration	usecond	227.71
L1/TEX Cache Throughput	%	73.24
L2 Cache Throughput	%	66.05
SM Active Cycles	cycle	266494.39
Compute (SM) Throughput	%	89.25

INF The kernel is utilizing greater than 80.0% of the available compute or memory performance of the device. To further improve performance, work will likely need to be shifted from the most utilized to another unit.

Start by analyzing workloads in the Compute Workload Analysis section.

Section: Memory Workload Analysis

Metric Name	Metric Unit	Metric Value
Memory Throughput	Tbyte/second	1.49
Mem Busy	%	68.53
Max Bandwidth	%	67.01
L1/TEX Hit Rate	%	0
L2 Hit Rate	%	62.12
Mem Pipes Busy	%	50.24

Section: GPU Speed Of Light Throughput

Metric Name	Metric Unit	Metric Value
DRAM Frequency	cycle/nsecond	2.61
SM Frequency	cycle/nsecond	1.47
Elapsed Cycles	cycle	8282
Memory Throughput	%	8.84
DRAM Throughput	%	1.50
Duration	usecond	5.60
L1/TEX Cache Throughput	%	23.84
L2 Cache Throughput	%	9.57
SM Active Cycles	cycle	2033.98
Compute (SM) Throughput	%	4.98

WRN This kernel grid is too small to fill the available resources on this device, resulting in only 0.1 full waves across all SMs. Look at Launch Statistics for more details.

Section: Memory Workload Analysis

Metric Name	Metric Unit	Metric Value
Memory Throughput	Gbyte/second	50.19
Mem Busy	%	8.84
Max Bandwidth	%	5.71
L1/TEX Hit Rate	%	0
L2 Hit Rate	%	90.40
Mem Pipes Busy	%	2.58

Practical Checklist: Will MPS Help My Workload?

- 1 Find your GPU's total SM count: `python3 -c "import torch; print(torch.cuda.get_device_properties(0).multi_processor_count)"`
- 2 Profile your ACTUAL workload with `nsys` to find which kernels dominate TOTAL GPU time (`nsys stats --report gpukernsum`)
- 3 For the top kernels by time%, check grid size (ALL 3 dimensions) and Block Limit via `ncu --section Occupancy`
- 4 Decision rule: $\text{total blocks} < \text{SM_count} \rightarrow$ real idle capacity, strong MPS candidate. $\text{Total blocks} \geq \text{SM_count} \rightarrow$ check Block Limit Registers/Shared Mem; if pinned at 1, MPS only gives the smaller context-switch-removal gain (~16%)
- 5 Skip all of the above and just A/B test it directly — empirical always wins over theoretical; use profiling to understand WHY, not as a substitute for measuring
- 6 LLM-specific caveat: MPS doesn't share GPU MEMORY, only compute/SM scheduling. Each process needs its own full weights + KV cache — you'll usually run out of memory before idle SM capacity. MPS helps most for co-locating several SMALL models or many low-batch replicas.

Why Training Needs Far More Memory Than Inference

- Inference only needs the model's weights (plus a small activation footprint for the current forward pass)
- Training needs FOUR categories of GPU memory simultaneously:



Parameters

The weights themselves — same as inference



Gradients

One value per parameter, computed during backward()



Optimizer States

Adam's extra bookkeeping per parameter — the big one, covered next



Activations

Intermediate layer outputs kept for backward() — scales with batch size $\times$ sequence length

How the Adam Optimizer Works

- Adam keeps a running (exponential moving average) estimate of each gradient's mean AND variance, then uses both to adapt the step size per-parameter

```

m_t = β1 · m_{t-1} + (1 - β1) · g_t           # 1st moment (momentum) — EMA of the gradient
v_t = β2 · v_{t-1} + (1 - β2) · g_t^2         # 2nd moment (variance) — EMA of the squared gradient

m_hat_t = m_t / (1 - β1^t)                    # bias correction (EMAs start biased toward 0)
v_hat_t = v_t / (1 - β2^t)

θ_t = θ_{t-1} - lr · m_hat_t / (√v_hat_t + ε)  # the actual parameter update

```

The memory-relevant detail: m_t and v_t are NOT scratch values — they must PERSIST across every training step, one value per parameter, for the whole run. That's two full extra copies of the parameter tensor living in GPU memory the entire time.

Why Adam Needs ~3x More Memory

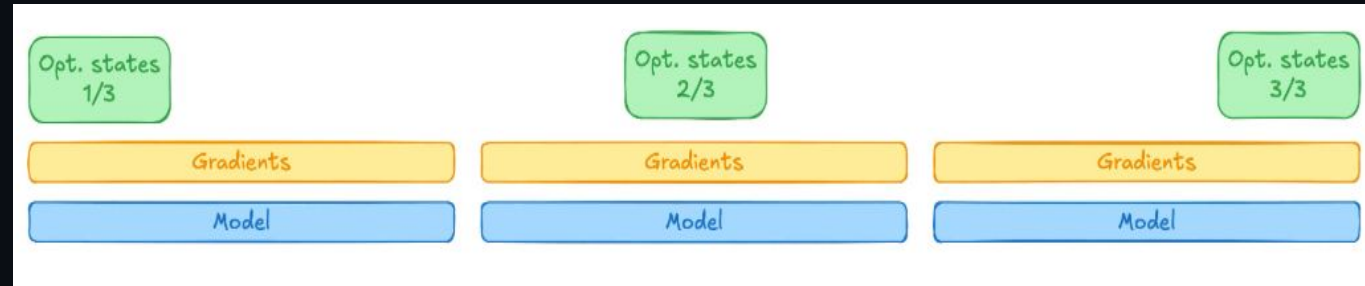
Standard mixed-precision training memory breakdown, per parameter:

Buffer	Precision	Bytes / Param
Parameters (used in fwd/bwd)	fp16	2
Gradients	fp16	2
Master copy of parameters	fp32 (optimizer state)	4
Adam momentum (m)	fp32 (optimizer state)	4
Adam variance (v)	fp32 (optimizer state)	4
TOTAL		16



The “3x”: the master weights, momentum, and variance are THREE separate fp32 buffers, each the same size as the fp32 parameter count (4 bytes/param) — 12 of the 16 bytes/param total exist purely for the optimizer's bookkeeping, 3× the size of the parameters they're optimizing. This exact 16-byte/param figure is the basis for DeepSpeed's ZeRO memory formulas on the next slides.

DeepSpeed ZeRO: Stage 1

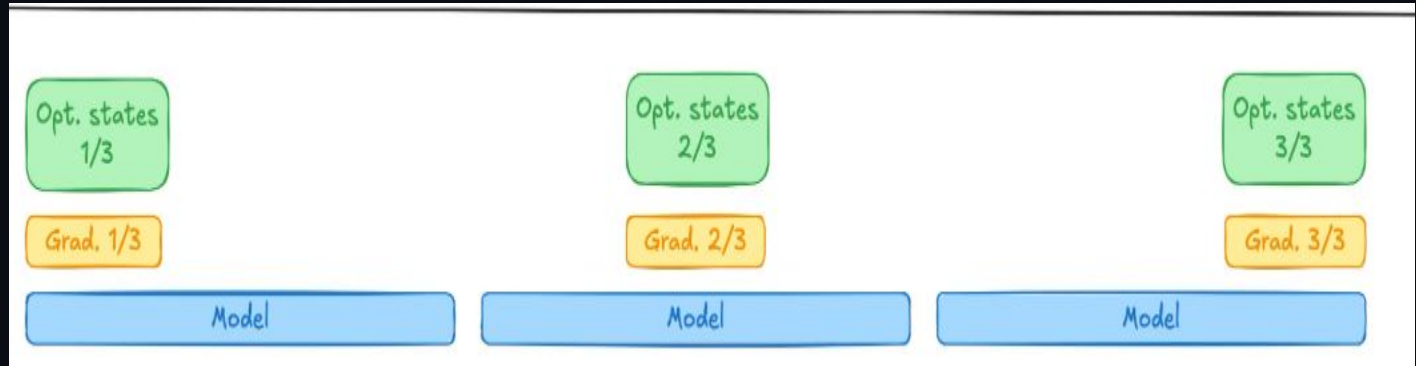


The core idea: standard data parallelism replicates ALL of params + gradients + optimizer states on EVERY GPU. ZeRO instead SHARDS (partitions) these buffers across the N data-parallel GPUs, reconstructing only what's needed, when it's needed.

Stage 1 (Pos): Optimizer State Partitioning

- Process with batch of 64 and 4 GPUs
 - Batch is divided into 4 parts of 16 data points each
 - Each sub-batch goes to 1 GPU
 - GPU does forward & backwards locally to compute the gradients.
 - Gradients are then AllReduce to send to all the GPUs -> avg(gradients on different machines)
 - optimizer.step() will update the weights for which that GPU has optimizer state. That means only $\frac{1}{4}$ of parameters are updated on each GPU, rest are old same.
 - Each GPU broadcast its updated parameters.
- Data Transfer happen Aysnchrously using DMA, SMs continue to work on kernels without break.
 - Let say there are 4 layers, as soon as backward process layer 4 and compute its gradient, gradients of layer 4 are transferred asynchronously without cuda kernels need to wait for anything. So it start the backward step of layer 3 and so on.
 - Same thing happen when all the weights are transferred, as soon as some final weights are computed they are transferred.
- Since the transfer happen aysnchously its mostly free.

DeepSpeed ZeRO: Stage 2



Stage 1 (Pos): Optimizer State Partitioning

- Process with batch of 64 and 4 GPUs
 - Very similar to Stage 1 with additional communication of sending the gradients as soon as we compute them.

DeepSpeed ZeRO: Stage 3



Stage 1 (Pos): Optimizer State Partitioning

- Process with batch of 64 and 4 GPUs
 - Each GPU has only $\frac{1}{4}$ of each layer weights.
 - Batch is divided into 4 parts of 16 data points each
 - Each sub-batch goes to 1 GPU
 - GPU does forward & backwards locally to compute the gradients. But GPU doesn't have full weights. GPU does AllGather operation before running forward pass of that layer and immediately release the weights after the forward pass. Similarly, GPU does AllGather before running the backward pass on that layer and immediately release the memory after doing the backward on that layer.
 - After doing the backward pass, process is exactly same as Stage 2.
- We prefetch the weights of the next layer to execute but since weights are transferred which are required in the next layer, it is possible that the execution have to wait for weights to arrive.

How Prefetch + Overlap Actually Work

- DeepSpeed issues the all-gather for layer $i+1$'s parameters on a SEPARATE CUDA/NCCL stream from the compute stream — the two streams run truly concurrently on the GPU
- While the compute stream is busy with layer i 's forward (or backward) math, the communication stream is already moving layer $i+1$'s parameter shards over NVLink in the background
- By the time compute finishes layer i , layer $i+1$'s parameters have (ideally) already landed — zero stall before starting layer $i+1$
- Backward pass mirrors this: once a layer's gradients are computed, they're REDUCE-SCATTERED asynchronously while backward compute continues into the layers before it

Overlapped execution (`overlap_comm = true`)



Gather 2 runs DURING Compute 1; Gather 3 runs DURING Compute 2. No orange block sits alone on the critical path.

Removing DRAM bottleneck using Quantization

Storing each weight in fewer bits — memory scales directly with bits/param:

Format	Bytes / Param	7B Model Size	Typical Use
fp16 / bf16	2	~14 GB	Standard training/inference baseline
int8	1	~7 GB	Simple post-training quantization
GPTQ / AWQ int4	0.5	~3.5 GB	Aggressive PTQ, near-fp16 accuracy

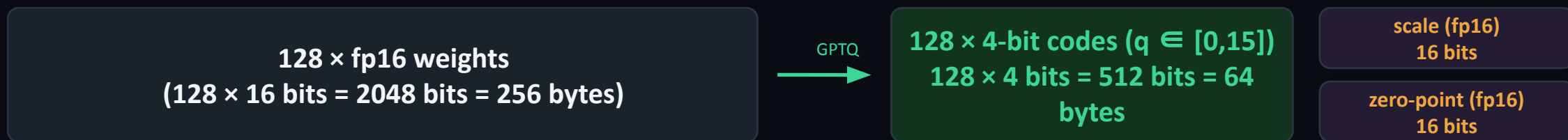


Why it's more than “fits on a smaller GPU”: decode is memory-bandwidth-bound (the roofline discussion earlier in this deck) — every generated token re-reads the full weight matrix from HBM. Fewer bytes per weight means fewer bytes moved per token, which speeds up decode directly, not just the model's footprint.

Method	One-line distinction
GPTQ	Layer-by-layer, Hessian-guided PTQ — covered in depth next
AWQ	Protects the small % of “salient” weight channels identified from activation magnitude, quantizes the rest
GGUF / llama.cpp, bitsandbytes NF4	Simpler blockwise quantization schemes, popular for CPU/edge and QLoRA fine-tuning

GPTQ: How the 4-Bit Weights Are Actually Packed

Weights are quantized in GROUPS (default group_size = 128), each group sharing one scale and zero-point:



+ ONE scale and ONE zero-point per group of 128 — not per weight

```
dequantize:  w_i  ≈  scale × (q_i) - zero_point           for i = 1 .. 128 in the group
```

Important: Since the format of the weights are changed and now we need to transform it back to 16 bits before doing actual matrix multiplication, the packing format also changes.

GPTQ: The Core Idea

The objective, per layer: find quantized weights $\hat{W}$ that keep this layer's OUTPUT as close as possible to the original — not just weights that individually look close to the original values.

$$\operatorname{argmin}_{\hat{W}} ||W X - \hat{W} X||^2 \quad (X = \text{real calibration activations for this layer, } \sim 128 \text{ samples})$$

- $H = 2XX^T + \lambda I$ — the layer's Hessian, built from the SAME calibration activations X (λ is a small damping term for numerical stability)
- H captures which weights matter most for this layer's output — not just “big weight = important,” but “big effect on the output, given how inputs actually vary”

$$\delta = - (w_q - \text{quant}(w_q)) / [H^{-1}]_{qq} \cdot H^{-1}_{\{:,q\}} \quad (\text{the Optimal-Brain-Surgeon correction step})$$

In plain terms: quantizing weight w_q introduces an error. This formula immediately spreads that error across the weights NOT YET quantized in the same row, weighted by H^{-1} — weights closely coupled (via the Hessian) to w_q absorb more of the correction, so the layer's overall output barely moves.

Part Three: Key Takeaways



HBM bottleneck diagnosis: Escalate from nvidia-smi dmon → ncu Speed Of Light → raw dram__bytes metrics → DCGM for production — same escalating-precision workflow as SM occupancy.



Adam's memory cost: 3 extra fp32 buffers (master weights, momentum, variance) at 4 bytes/param each — 12 of 16 bytes/param exist just for the optimizer.



ZeRO stages: 1 shards optimizer states, 2 adds gradients (the practical default), 3 adds parameters (linear scaling, higher comm cost).



Async overlap: Prefetching the next layer's parameters on a separate stream hides all-gather/reduce-scatter latency behind ongoing compute — verified the same way as the MPS kernel-overlap proof.

Every technique here follows the same discipline as the rest of this deck: measure with the right tool, don't assume — then verify the fix actually did what the theory predicted.